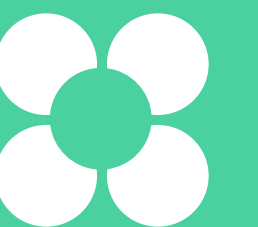


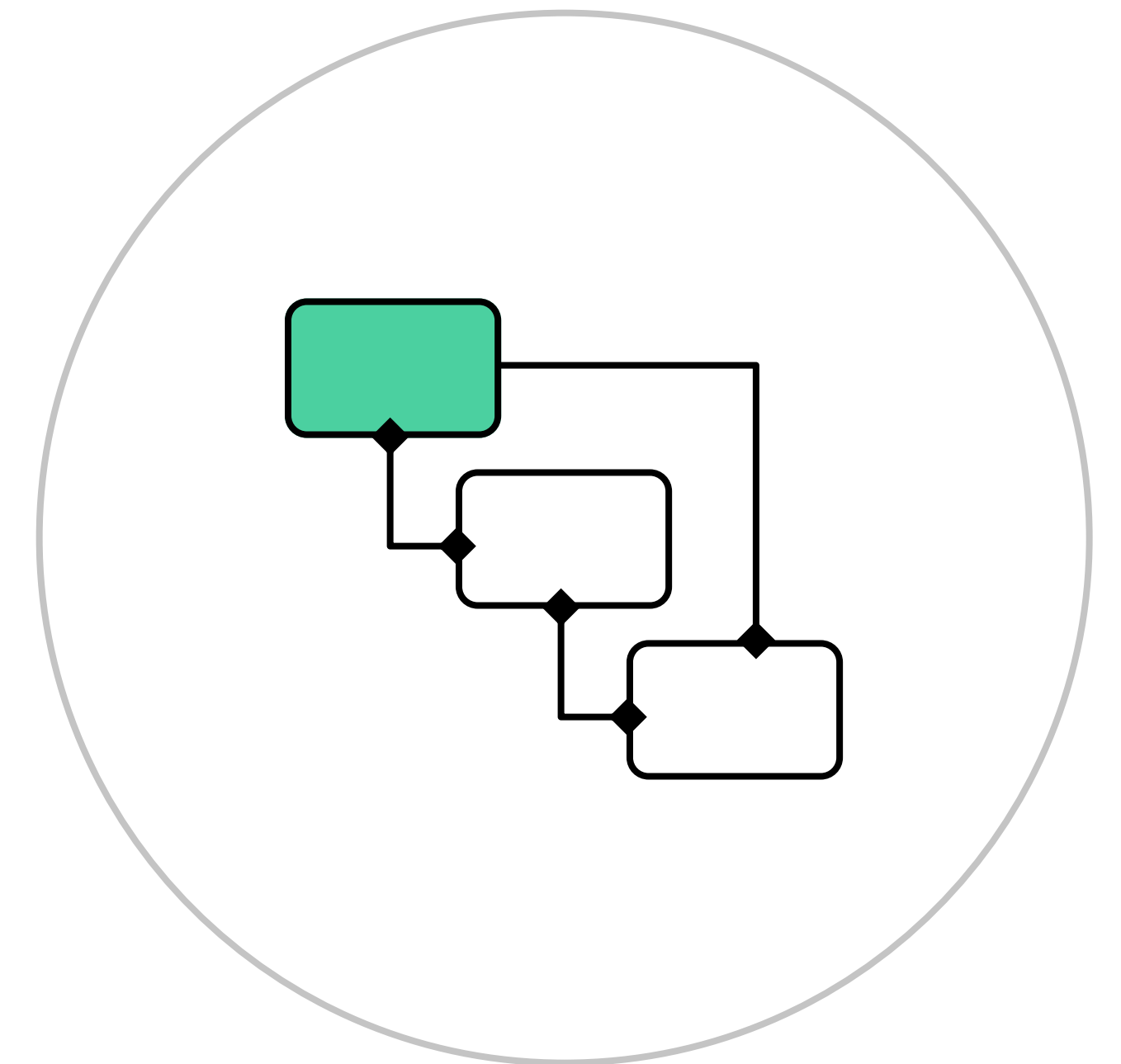
Методологии разработки

Филипп Воронов
Тимлид, поиск VK



Test driven development

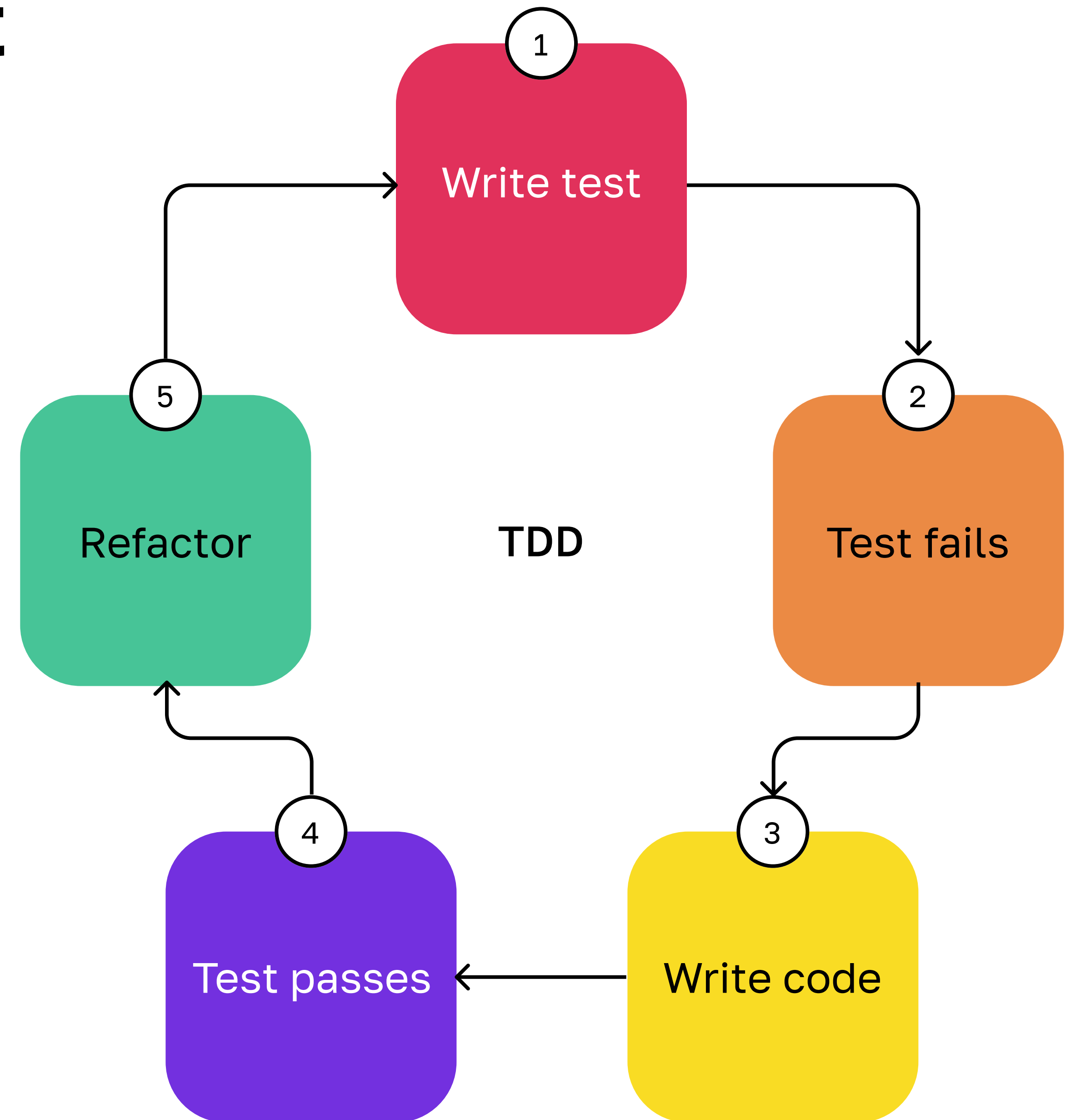
Test driven development — техника разработки ПО, основанная на повторении коротких итераций цикла, которые начинаются с написания тестов



Test driven development

Этапы разработки:

- 1 Написать тесты на новый набор функций
- 2 Запустить тесты и проверить, что новые тесты не проходят (нет функций)
- 3 Написать код нового набора функций
- 4 Повторно запустить тесты. Убедиться, что тесты стали проходить
- 5 Выполнить рефакторинг кода



Test driven development

Первый этап — напомним простейший калькулятор, который сможет складывать и вычитать целые числа:

```
public class TestCalculator {  
    Random random = new Random();  
    @Test  
    public void testAddition() {  
        Calculator calculator = new Calculator();  
        int var1 = random.nextInt();  
        int var2 = random.nextInt();  
        int sumResult = calculator.addition(var1, var2);  
        assertTrue("Неверное сложение " + var1 + " и " + var2, var1 + var2 == sumResult);  
    }  
    @Test  
    public void testSubstraction() {  
        Calculator calculator = new Calculator();  
        int var1 = random.nextInt();  
        int var2 = random.nextInt();  
        int subResult = calculator.substraction(var1, var2);  
        assertTrue("Неверное вычитание " + var1 + " и " + var2, var1 - var2 == subResult);  
    }  
}
```

Test driven development

Второй этап — пока нет реализации, запуск тестов будет заканчиваться провалом:

```
ooo
java.lang.AssertionError: Неверное сложение  -966727712 и 856128038
+ <2 internal calls>
+   at ru.net.metodology.TestCalculator.testAddition(TestCalculator.java:19) <19 internal calls>
   at com.intellij.rt.junit.IdeaTestRunner$Repeater.startRunnerWithArgs(IdeaTestRunner.java:33)
   at com.intellij.rt.junit.JUnitStarter.prepareStreamsAndStart(JUnitStarter.java:230)
   at com.intellij.rt.junit.JUnitStarter.main(JUnitStarter.java:58)

java.lang.AssertionError: Неверное вычитание  869669172 и 140629519
+ <2 internal calls>
+   at ru.net.metodology.TestCalculator.testSubstraction(TestCalculator.java:28) <19 internal calls>
   at com.intellij.rt.junit.IdeaTestRunner$Repeater.startRunnerWithArgs(IdeaTestRunner.java:33)
   at com.intellij.rt.junit.JUnitStarter.prepareStreamsAndStart(JUnitStarter.java:230)
   at com.intellij.rt.junit.JUnitStarter.main(JUnitStarter.java:58)

Process finished with exit code -1
```

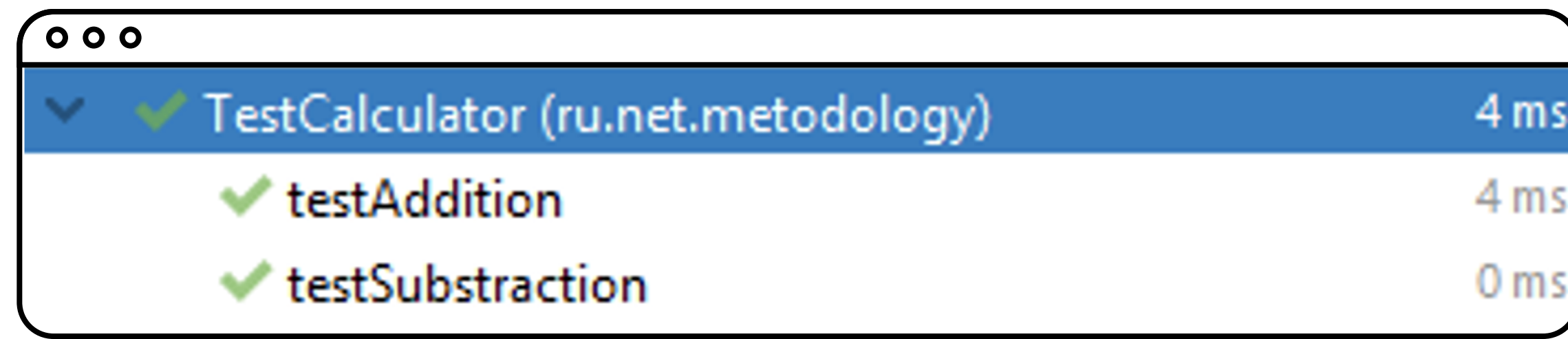
Test driven development

Третий этап — напомним реализацию:

```
public class Calculator {  
  
    public int addition(int var1, int var2) {  
        return var1 + var2;  
    }  
  
    public int subtraction(int var1, int var2) {  
        return var1 - var2;  
    }  
}
```

Test driven development

Четвёртый этап — повторно запустим тесты, чтобы убедиться, что функции реализованы верно:



A screenshot of a test runner window with a title bar containing three small circles. The window displays a list of test results. The first item is 'TestCalculator (ru.net.metodology)' with a green checkmark and a duration of '4 ms'. Below it are two sub-items: 'testAddition' with a green checkmark and '4 ms', and 'testSubtraction' with a green checkmark and '0 ms'.

TestCalculator (ru.net.metodology)	4 ms
testAddition	4 ms
testSubtraction	0 ms

Пятый этап — рефакторинг

Test driven development

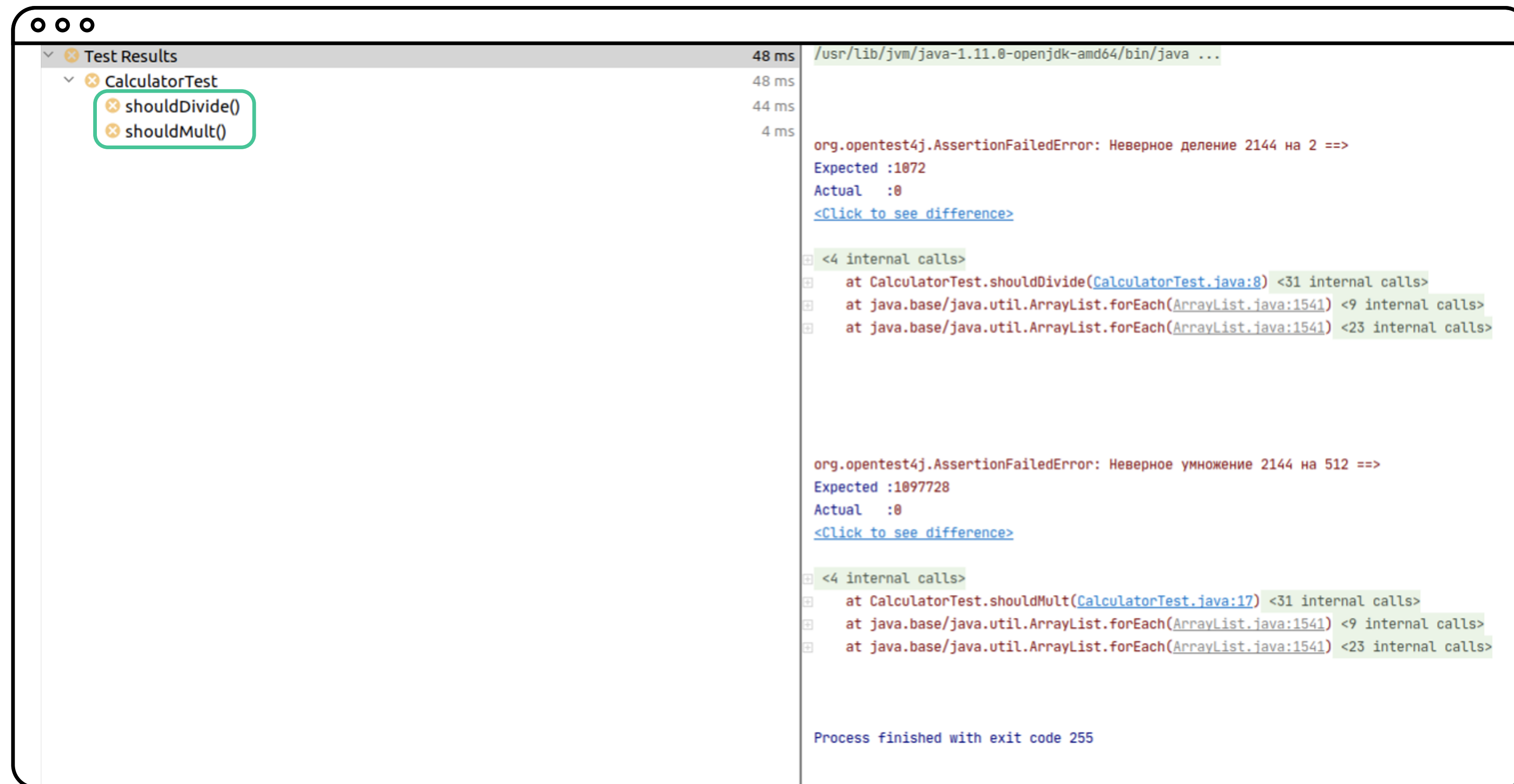
Напишем тесты для нового набора функций:

```
@Test
public void testDivide() {
    Calculator calculator = new Calculator();
    int var1 = random.nextInt();
    int var2 = random.nextInt();
    int subResult = calculator.divide(var1, var2);
    assertTrue("Неверное деление " + var1 + " на " + var2, var1 / var2 == subResult);
}

@Test
public void testMultiply() {
    Calculator calculator = new Calculator();
    int var1 = random.nextInt();
    int var2 = random.nextInt();
    int subResult = calculator.multiply(var1, var2);
    assertTrue("Неверное вычитание " + var1 + " на " + var2, var1 * var2 == subResult);
}
}
```

Test driven development

Запустим все тесты и увидим, что тесты на новый набор функций не прошли, а старые проходят:



The screenshot shows the 'Test Results' window in IntelliJ IDEA. The 'Test Results' tree on the left shows a failed test 'CalculatorTest' with two sub-items: 'shouldDivide()' and 'shouldMult()'. The 'shouldDivide()' test is highlighted with a green box. The main pane on the right displays the test execution details for 'shouldDivide()'. It shows an 'AssertionFailedError' with the message 'Неверное деление 2144 на 2 ==>'. The 'Expected' value is '1072' and the 'Actual' value is '0'. The stack trace shows the test method 'shouldDivide()' in 'CalculatorTest.java:8' and the internal calls to 'ArrayList.forEach()' in 'ArrayList.java:1541'. The 'shouldMult()' test is also shown as failed with a similar error message: 'Неверное умножение 2144 на 512 ==>'. The stack trace for 'shouldMult()' shows the test method 'shouldMult()' in 'CalculatorTest.java:17' and the internal calls to 'ArrayList.forEach()' in 'ArrayList.java:1541'. The process finished with exit code 255.

```
Test Results
  CalculatorTest
    shouldDivide()
    shouldMult()

/usr/lib/jvm/java-11.0-openjdk-amd64/bin/java ...

org.opentest4j.AssertionFailedError: Неверное деление 2144 на 2 ==>
Expected :1072
Actual   :0
<Click to see difference>

<4 internal calls>
  at CalculatorTest.shouldDivide(CalculatorTest.java:8) <31 internal calls>
  at java.base/java.util.ArrayList.forEach(ArrayList.java:1541) <9 internal calls>
  at java.base/java.util.ArrayList.forEach(ArrayList.java:1541) <23 internal calls>

org.opentest4j.AssertionFailedError: Неверное умножение 2144 на 512 ==>
Expected :1097728
Actual   :0
<Click to see difference>

<4 internal calls>
  at CalculatorTest.shouldMult(CalculatorTest.java:17) <31 internal calls>
  at java.base/java.util.ArrayList.forEach(ArrayList.java:1541) <9 internal calls>
  at java.base/java.util.ArrayList.forEach(ArrayList.java:1541) <23 internal calls>

Process finished with exit code 255
```

Test driven development

Напишем реализацию и запустим тесты:

```
public int divide(int var1, int var2) {  
    return var1 / var2;  
}  
  
public int multiply(int var1, int var2) {  
    return var1 * var2;  
}
```

TestCalculator (ru.net.metodology)	13 ms
testAddition	13 ms
testSubstraction	0 ms
testDivide	0 ms
testMultiply	0 ms

Test driven development

Преимущества:

- ⊕ снижение количества ошибок ПО
- ⊕ низкие риски при рефакторинге и внесении новых фич
- ⊕ легко тестируемый код
- ⊕ гибкий код с хорошей расширяемостью

Test driven development

Недостатки:

- не всё можно протестировать (часть нефункциональных требований)
- TDD предполагает использование только модульных тестов и моков
- требуется больше времени
- влияет человеческий фактор